

1 算法库

1.1 概述

ACH512 算法库包含 HRNG、DES、AES、SM1、SM4、SSF33、PKI、RSA、SM2_SM3、ECDSA、SHA1、SHA256、SHA384_512 等算法，通过调用 ROM 中相关函数实现算法初始化、加解密等算法功能。

算法模块中使用的数据格式如下：

```
typedef unsigned      char  UINT8;

typedef unsigned      int   UINT32;
```

1.2 HRNG

1.2.1 主要特性

- 内含可靠噪声振荡器；
- 符合国际 FIPS-140-2 和 NIST SP800-22 测试标准；
- 符合国密局《随机数检测规范》测试标准；

1.2.2 库需求说明

HRNG 模块需要的库及头文件如下：

hrng.lib	实现取随机数操作
hrng.h	hrng.lib 对应的头文件

HRNG 库需要的资源如下：

库大小	60(字节)
全局变量	无
堆栈大小	208(字节)

1.2.3 函数说明

1.2.3.1 随机数启动函数

函数形式	void hrng_initial(void);	
功能描述	启动随机数模块	
参数说明	输入	无
	输出	无
	返回值	无

1.2.3.2 取随机数函数(8 位)

函数形式	UINT8 get_hrng8(void);	
功能描述	取 8 位随机数	
参数说明	输入	无
	输出	无
	返回值	8 位随机数

1.2.3.3 取随机数函数(32 位)

函数形式	UINT32 get_hrng32(void);	
功能描述	取 32 位随机数	
参数说明	输入	无
	输出	无
	返回值	32 位随机数

1.2.3.4 取随机数函数(16 字节)

函数形式	void get_hrng_16bytes(UINT8 *hdata);	
功能描述	取 16 个字节随机数	
参数说明	输入	无
	输出	UINT8 *hdata: 存放随机数的起始地址
	返回值	无

1.2.3.5 取随机数函数(多个字节)

函数形式	UINT8 get_hrng(UINT8 *hdata, UINT32 byte_len);	
功能描述	取多个字节随机数	
参数说明	输入	UINT32 byte_len: 取随机数的字节长度
	输出	UINT8 *hdata: 存放随机数的起始地址
	返回值	0: 取随机数成功; 1: 取随机数失败

1.2.4 注意事项

- 1) 需要确保模块时钟使能寄存器中的 UAC，HRNG，HRNG 慢时钟模块时钟使能。

1.3 DES

1.3.1 主要特性

- 符合 FIPS 46-3 的加解密标准；
- 支持 DES 和 3DES 加解密运算；
- 支持 DES 算法 64（56）位密钥；
- 支持 3DES 算法 128（112）位和 192（168）位密钥；
- 支持 ECB（Electronic Code Book）模式和 CBC（Cipher Block Chaining）模式；
- 数据输入和输出支持 SWAP 模式，即大小端可配置；

1.3.2 库需求说明

DES 模块需要的库及头文件如下：

des.lib	实现 des 相关运算
des.h	des.lib 对应的头文件

DES 库需要的资源如下：

库大小	772(字节)
全局变量	无
堆栈大小	128(字节)

1.3.3 函数说明

1.3.3.1 DES 初始化函数

函数形式	void des_set_key(UINT8 key_num, UINT32 *keys, UINT8 swap_en);	
功能描述	DES 初始化(配置 DES 密钥及数据对调模式)	
参数说明	输入	UINT8 key_num: des 密钥个数 0x01: 单密钥 0x02: 双密钥 0x03: 三密钥
		UINT32 *keys: 存放密钥的起始地址

		UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
	输出	无
	返回值	无

1.3.3.2 DES 加解密函数

函数形式	<pre> UINT32 des_crypt(UINT32 *indata, UINT32 *outdata, UINT32 block_len, UINT8 operation, UINT8 mode, UINT32 *iv, UINT32 security_mode); </pre>	
功能描述	DES 加解密函数	
参数说明	输入	UINT32 *indata: 存放输入数据的起始地址
		UINT32 block_len: 加解密的块(64 bit)个数
		UINT8 operation:加解密操作选择 0: 加密 1: 解密
		UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
		UINT32 *iv: 存放 CBC 模式初始向量的起始地址
		UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
	输出	UINT32 *outdata: 存放输出结果的起始地址
	返回值	UINT32 加解密运行结果 0: 运行失败 0x5a9ada68: 运行成功

1.3.1 注意事项及数据格式

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, DES 模块时钟使能;
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, DES, HRNG, HRNG 慢时钟模块时钟使能;
- 3) 输入 DES 模块进行加解密运算的数据均为 32bit 字数组, 高 32 位字数据存储在数组的低位元素中, 每个字以大端方式存放, 举例如下:

设待加密数据为：0x112233445566778899aabbccddeeff00

则输入 DES 模块进行运算的数组为：

UINT32 plain_text[4] = {0x11223344, 0x55667788, 0x99aabbcc, 0xddeeff00};

如果使能 SWAP 模式，则输入 DES 模块进行运算的数组为：

UINT32 plain_text[4] = {0x44332211, 0x88776655, 0xccbbaa99, 0x00ffeedd};

输出数据格式与输入相同。

4) SWAP 模式对密钥、初始向量、输入数据、输出数据同时有效。

1.4 SM1

1.4.1 主要特性

- 支持国密 SM1 加解密算法；
- 支持数据密钥长度 128 比特，系统密钥长度 128 比特；
- 支持 128 比特明文和密文长度；
- 支持 ECB、CBC 工作模式；
- 数据输入和输出支持 SWAP 模式，即大小端可配置；

1.4.2 库需求说明

SM1 模块需要的库及头文件如下：

sm1.lib	实现 sm1 相关运算
sm1.h	sm1.lib 对应的头文件

SM1 库需要的资源如下：

库大小	848(字节)
全局变量	无
堆栈大小	204(字节)

1.4.3 函数说明

1.4.3.1 SM1 初始化函数

函数形式	void sm1_set_key(UINT32 *keyin, UINT8 sk, UINT8 swap_en);	
功能描述	SM1 初始化(配置密钥，数据对调模式及参数选择)	
参数说明	输入	UINT32 *keyin: 存放密钥的起始地址

		UINT8 sk: 参数选择 0: 内部参数 1: 外部参数
		UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
	输出	无
	返回值	无

1.4.3.2 SM1 加解密函数

函数形式	<pre> UINT32 sm1_crypt(UINT32 *indata, UINT32 *outdata, UINT32 block_len, UINT8 operation, UINT8 mode, UINT32 *iv, UINT32 security_mode); </pre>	
功能描述	SM1 加解密函数	
参数说明	输入	UINT32 *indata: 存放输入数据的起始地址
		UINT32 block_len: 加解密的块(128 bit)个数
		UINT8 operation:加解密操作选择 0: 解密 1: 加密
		UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
		UINT32 *iv: 存放 CBC 模式初始向量的起始地址
		UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
	输出	UINT32 *outdata: 存放输出结果的起始地址
	返回值	UINT32 加解密运行结果 0: 运行失败 0x5aaada6e: 运行成功

1.4.4 注意事项及数据格式

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, SM1 模块时钟使能
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, SM1, HRNG, HRNG 慢时

钟模块时钟使能;

- 3) 输入 SM1 模块进行加解密运算的数据均为 32bit 字数组，高 32 位字数据存储在数组的低位元素中，每个字以大端方式存放，举例如下：

设待加密数据为：0x112233445566778899aabbccddeeff00

则输入 SM1 模块进行运算的数组为：

```
UINT32 plain_text[4] = {0x11223344, 0x55667788, 0x99aabbcc, 0xddeeff00};
```

如果使能 SWAP 模式，则输入 SM1 模块进行运算的数组为：

```
UINT32 plain_text [4]= {0x44332211, 0x88776655, 0xccbbaa99, 0x00feedd};
```

输出数据格式与输入相同。

- 4) SWAP 模式对密钥、初始向量、输入数据、输出数据同时有效。

1.5 SM4

1.5.1 主要特性

- 支持国密 SM4 加密和解密运算；
- 支持 Electronic Code Book (ECB)模式和 Cipher Block Chaining (CBC)模式；
- 数据输入和输出支持 SWAP 模式，即大小端可配置；

1.5.2 库需求说明

SM4 模块需要的库及头文件如下：

sm4.h	实现 sm4 相关运算
sm4.lib	sm4.lib 对应的头文件

SM4 库需要的资源如下：

库大小	808(字节)
全局变量	无
堆栈大小	204(字节)

1.5.3 函数说明

1.5.3.1 SM4 初始化函数

函数形式	void sm4_set_key(UINT32 *keyin, UINT8 swap_en);
功能描述	SM4 初始化(配置密钥及数据对调模式)

参数说明	输入	UINT32 *keyin: 存放密钥的起始地址
		UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
	输出	无
	返回值	无

1.5.3.2 SM4 加解密函数

函数形式	<pre> UINT32 sm4_crypt(UINT32 *indata, UINT32 *outdata, UINT32 block_len, UINT8 operation, UINT8 mode, UINT32 *iv, UINT32 security_mode); </pre>	
功能描述	SM4 加解密函数	
参数说明	输入	UINT32 *indata: 存放输入数据的起始地址
		UINT32 block_len: 加解密的块(128 bit)个数
		UINT8 operation:加解密操作选择 0: 解密 1: 加密
		UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
		UINT32 *iv: 存放 CBC 模式初始向量的起始地址
		UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
	输出	UINT32 *outdata: 存放输出结果的起始地址
	返回值	UINT32 加解密运行结果 0: 运行失败 0xa59ada68: 运行成功

1.5.4 注意事项及数据格式

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC，EMW，SM4 模块时钟使能；
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC，EMW，SM4，HRNG，HRNG 慢时钟模块时钟使能；
- 3) 输入 SM4 模块进行加解密运算的数据均为 32bit 字数组，高 32 位字数据存储在数组的

低位元素中，每个字以大端方式存放，举例如下：

设待加密数据为：0x112233445566778899aabbccddeeff00

则输入 SM4 模块进行运算的数组为：

UINT32 plain_text[4] = {0x11223344, 0x55667788, 0x99aabbcc, 0xddeeff00};

如果使能 SWAP 模式，则输入 SM4 模块进行运算的数组为：

UINT32 plain_text [4]= {0x44332211, 0x88776655, 0xccbbaa99, 0x00ffeedd};

输出数据格式与输入相同。

- 4) SWAP 模式对密钥、初始向量、输入数据、输出数据同时有效。

1.6 AES

1.6.1 主要特性

- 支持 AES 加密和解密运算；
- 支持 ECB 模式和 CBC 模式；
- 支持 128/192/256 bit 密钥长度；
- 数据输入和输出支持 SWAP 模式，即大小端可配置；

1.6.2 库需求说明

AES 模块需要的库及头文件如下：

aes.lib	实现 aes 相关运算
aes.h	aes.lib 对应的头文件

AES 库需要的资源如下：

库大小	844(字节)
全局变量	无
堆栈大小	212(字节)

1.6.3 函数说明

1.6.3.1 AES 初始化函数

函数形式	aes_set_key(UINT32 *keyin, UINT8 key_len, UINT8 swap_en);	
功能描述	AES 初始化(配置密钥，密钥长度，数据对调模式)	
	输入	UINT32 *keyin: 存放密钥的起始地址

参数说明	输入	UINT8 key_len: 密钥长度选择长度选择 0:128 1:192 2:256
		UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
	输出	无
	返回值	无

1.6.3.2 AES 加解密函数

函数形式	<pre> UINT32 aes_crypt(UINT32 *indata, UINT32 *outdata, UINT32 block_len, UINT8 operation, UINT8 mode, UINT32 *iv, UINT32 security_mode); </pre>	
功能描述	AES 加解密函数	
参数说明	输入	*indata: 存放输入数据的起始地址
		UINT32 block_len: 加解密的块(128 bit)个数
		UINT8 operation:加解密操作选择 0: 解密 1: 加密
		UINT8 mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
		UINT32 *iv: 存放 CBC 模式初始向量的起始地址
		UINT32 security_mode: 安全模式选择 0x12345678: 普通模式 0: 安全模式
	输出	UINT32 *outdata: 存放输出结果的起始地址
	返回值	UINT32 加解密运行结果 0: 运行失败 0xa59ada68: 运行成功

1.6.4 注意事项及数据格式

- 1) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, AES 模块时钟使能;
- 2) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, AES, HRNG, HRNG 慢时

钟模块时钟使能；

3) 输入 AES 模块进行加解密运算的数据均为 32bit 字数组，高 32 位字数据存储在数组的低位元素中，每个字以大端方式存放，举例如下：

设待加密数据为：0x112233445566778899aabbccddeeff00

则输入 AES 模块进行运算的数组为：

UINT32 plain_text[4] = {0x11223344, 0x55667788, 0x99aabbcc, 0xddeeff00};

如果使能 SWAP 模式，则输入 AES 模块进行运算的数组为：

UINT32 plain_text [4]= {0x44332211, 0x88776655, 0xccbbaa99, 0x00ffeedd};

输出数据格式与输入相同。

SWAP 模式对密钥、初始向量、输入数据、输出数据同时有效。

1.7 SSF33

1.7.1 主要特性

- 支持国密 SSF33 加解密算法；
- 支持数据密钥长度 128 比特，系统密钥长度 128 比特；
- 支持 128 比特明文和密文长度；
- 支持 ECB、CBC 工作模式；
- 支持内部参数和外部参数两种模式；
- 数据输入和输出支持 SWAP 模式，即大小端可配置；

1.7.2 库需求说明

SSF33 模块需要的库及头文件如下：

ssf33.lib	实现 ssf33 相关运算
ssf33.h	ssf33.lib 对应的头文件

SSF33 库需要的资源如下：

库大小	840(字节)
全局变量	无
堆栈大小	308(字节)

1.7.3 函数说明

1.7.3.1 SSF33 初始化函数

函数形式	void ssf33_set_key(UINT32 se_sel,UINT32 *para, UINT32 *ks,UINT32 swap);	
功能描述	SSF33 初始化(配置密钥, 数据对调模式及参数选择)	
参数说明	输入	UINT32 se_sel: 参数选择 0: 内部参数 1: 外部参数
		UINT32 *para: 存放参数的起始地址
		UINT32 *ks: 存放秘钥的起始地址
		UINT8 swap_en: 输入输出数据大小端对调使能 0: 输入输出数据大小端对调禁止 1: 输入输出数据大小端对调使能
	输出	无
	返回值	无

1.7.3.2 SSF33 加解密函数

函数形式	UINT32 ssf33_crypt_data(UINT32 * datain, UINT32 * dataout, UINT32 block_len, UINT32 enc, UINT32 data_mode, UINT32 iv[], UINT32 security_mode);	
功能描述	SSF33 加解密函数	
参数说明	输入	UINT32 * datain: 存放输入数据的起始地址
		UINT32 block_len: 加解密的块(128 bit)个数
		UINT32 enc:加解密操作选择 0: 解密 1: 加密
		UINT32 data_mode: 加解密模式选择 0: ECB 模式 1: CBC 模式
		UINT32 *iv: 存放 CBC 模式初始向量的起始地址
		UINT32 security_mode: 安全模式选择 0x5a3489a5: 普通模式 0: 安全模式
	输出	UINT32 *outdata: 存放输出结果的起始地址

	返回值	UINT32 加解密运行结果 0: 运行失败 0xabef5a58: 运行成功
--	-----	---

1.7.4 注意事项及数据格式

- 4) 普通模式需要确保模块时钟使能寄存器中的 UAC, EMW, SSF33 模块时钟使能;
- 5) 安全模式需要确保模块时钟使能寄存器中的 UAC, EMW, SSF33, HRNG, HRNG 慢时钟模块时钟使能;
- 6) 输入 SSF33 模块进行加解密运算的数据均为 32bit 字数组, 高 32 位字数据存储在数组的低位元素中, 每个字以大端方式存放, 举例如下:

设待加密数据为: 0x112233445566778899aabbccddeeff00

则输入 SSF33 模块进行运算的数组为:

```
UINT32 plain_text[4] = {0x11223344, 0x55667788, 0x99aabbcc, 0xddeeff00};
```

如果使能 SWAP 模式, 则输入 SSF33 模块进行运算的数组为:

```
UINT32 plain_text [4]= {0x44332211, 0x88776655, 0xccbbaa99, 0x00ffeedd};
```

输出数据格式与输入相同。

- 7) SWAP 模式对密钥、初始向量、输入数据、输出数据同时有效。

1.8 RSA

1.8.1 主要特性

- 支持 64 到 4096 位的模乘、模平方、模加减、模幂和模逆运算操作;
- 支持 64 到 2048 位 CRT 密钥生成和加解密, 其中密钥生成的步长是 64 位;

1.8.2 库需求说明

RSA 库需要的库及头文件如下:

rsa_keygen.lib	实现 rsa 及 div 运算
rsa_keygen.h	支持 rsa 运算的头文件
divider.h	支持除法运算的头文件

RSA 需要的资源如下:

库大小	6648 (字节)
-----	-----------

全局变量	无
堆栈大小	1924（字节）

1.8.3 函数说明

1.8.3.1 模乘模幂运算

函数形式	<pre> UINT8 rsa_mul_me(UINT32 *a_data, UINT8 a_length, UINT32 *b_data, UINT8 b_length, UINT32 *n_data, UINT8 n_length, UINT32 *out_data, UINT8 *out_length, UINT8 mode); </pre>	
功能描述	模乘模幂运算(大数)	
参数说明	输入	UINT32 *a_data: 模幂底数或者模乘被乘数的起始地址
		UINT8 a_length: a_data 的字长度
		UINT32 *b_data: 模幂指数者模乘乘数的起始地址
		UINT8 b_length: b_data 的字长度
		UINT32 *n_data: 模数据的起始地址
		UINT8 n_length: n_data 的字长度
		UINT8 mode: 模式功能选择 0x01: 模乘 0x30: 模幂
	输出	UINT32 *out_data: 模幂或者模乘结果的起始地址
		UINT8* out_length: out_data 的字长度，长度是一个偶数
	返回值	0: 运算成功 1: 运算失败

1.8.3.2 模幂运算(CRT)

函数形式	UINT8 rsa_decrypt_CRT(UINT32 *in_data, UINT8 in_length, UINT32 *p_data, UINT8 p_length, UINT32 *q_data, UINT8 q_length, UINT32 *dp_data, UINT8 dp_length, UINT32 *dq_data, UINT8 dq_length, UINT32* qInv_data, UINT8 qInv_length, UINT32 *out_data, UINT8 *out_length, RSA_G_STR *p_rsa_str, MATH_G_STR *p_math_str, UINT32 *e_data, UINT8 e_length, UINT32 mode);	
功能描述	模幂运算(CRT 方式)	
参数说明	输入	UINT32 *in_data: 模幂底数的起始地址
		UINT8 in_length: in_data 的字长度
		UINT32 *p_data: p 的起始地址
		UINT8 p_length: p 的字长度
		UINT32 *q_data: q 的起始地址
		UINT8 q_length: q 的字长度
		UINT32 *dp_data: dp 的起始地址
		UINT8 dp_length: dp 的字长度
		UINT32 *dq_data: dq 的起始地址
		UINT8 dq_length: dq 的字长度
		UINT32 * qInv _data: qInv 的起始地址
		UINT8 qInv _length: qInv 的字长度
		RSA_G_STR *p_rsa_str: RSA_G_STR 结构指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT32 *e_data: 公钥 e 的起始地址
		UINT8 e_length: 公钥 e 的字长度
		UINT32 mode: 安全模式选择
		0x57D525A0: 普通模式
		其他数据: 安全模式

	输出	UINT32 *out_data: 模幂结果的起始地址
		UINT8* out_length: out_data 的字长度
	返回值	0: 运算成功; 1: 运算失败

1.8.3.3 RSA 密钥生成

函数形式	<pre> UINT8 RSA_keygen(RSA_KEYGEN_G_STR *p_rsa_keygen_str, MATH_G_STR *p_math_str, UINT8 nDigits); </pre>	
功能描述	RSA 密钥生成	
参数说明	输入	RSA_KEYGEN_G_STR*p_rsa_keygen_str: RSA_KEYGEN_G_STR 结构体指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 nDigits: 密钥生成的字长度
	输出	输出数据存放在 RSA_KEYGEN_G_STR 结构体指针对应的空间
	返回值	0: 运算成功; 1: 运算失败

1.8.3.4 RSA 密钥生成(CRT)

函数形式	<pre> UINT8 RSA_keygen_CRT(RSA_KEYGEN_G_STR *p_rsa_keygen_str, MATH_G_STR *p_math_str, UINT8 nDigits); </pre>	
功能描述	RSA 密钥生成(CRT)	
参数说明	输入	RSA_KEYGEN_G_STR*p_rsa_keygen_str: RSA_KEYGEN_G_STR 结构体指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 nDigits: 密钥生成的字长度
	输出	输出数据存放在 RSA_KEYGEN_G_STR 结构体指针对应的空间
	返回值	0: 运算成功; 1: 运算失败

1.8.4 注意事项及数据格式

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟模块时钟使能;
- 2) 通过时钟分频寄存器配置好系统时钟和算法时钟;
- 3) 使用私钥运算请调用 rsa_decrypt_CRT () 函数的安全模式;
- 4) 使用公钥运算可以调用 rsa_mul_me()函数;

5) RSA 的所有数据存放方式如下：

低 32 位字数据在数组的低位，每个字以大端方式存放。

若：A = 0x11223344556677889900；

则：A[0] = 0x77889900；

A[1] = 0x33445566；

A[2] = 0x00001122；

6) RSA_keygen 和 RSA_keygen_CRT 函数需要给结构体指针 p_rsa_keygen_str 中的变量按照密钥长度进行分配空间，具体可见 demo 工程中的 lib_variable_initial 函数。不需要给结构体指针 math_glb_str 分配空间。rsa_decrypt_CRT 函数也不需要给结构体指针 math_glb_str 和 rsa_glb_str 分配空间。

1.9 SM2_SM3

1.9.1 主要特性

- 支持国密 256 位的 SM2 密钥生成、加/解密、签名验证以及密钥交换；
- 支持国密 SM3 杂凑算法；

1.9.2 库需求说明

SM2_SM3 库需要的头文件如下：

sm2sm3.lib	实现 SM2SM3 相关运算
sm2.h	SM2 相关运算对应的头文件
sm3.h	SM3 相关运算对应的头文件

SM2_SM3 库需要的资源如下：

库大小	6348（字节）
全局变量	无
堆栈大小	1736（字节）

1.9.3 函数说明

1.9.3.1 SM2 签名运算

函数形式	UINT8 sm2_sign(ECC_G_STR *p_ecc_para, MATH_G_STR *p_math_str, UINT8 *hashdata, UINT8 *PrivateKey, UINT8 *Signature0, UINT8 *Signature1, UINT32 mode);	
功能描述	SM2 签名运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 *hashdata: 摘要的起始地址
		UINT8*PrivateKey: 私钥的起始地址
		UINT8 mode: 模式功能选择, 0xF27E4B40: 普通模式 其他数据: 安全模式
	输出	UINT8 *Signature0: 签名结果 r 的起始地址
		UINT8 *Signature1: 签名结果 s 的起始地址
	返回值	0: 运算成功; 其他: 运算失败

1.9.3.2 SM2 验签运算

函数形式	int sm2_verify(ECC_G_STR *p_ecc_para, MATH_G_STR *p_math_str, UINT8 *hashdata, UINT8 *PublicKey, UINT8 *Signature0, UINT8 *Signature1);	
功能描述	SM2 验签运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 *hashdata: 摘要的起始地址
		UINT8* PublicKey: 公钥的起始地址
		UINT8 *Signature0: 签名 s 的起始地址
		UINT8 *Signature1: 签名 r 的起始地址
	输出	无

	返回值	0：验签成功；其他：验签失败
--	-----	----------------

1.9.3.3 SM2 加密运算

函数形式	<pre> UINT8 sm2_encrypt(ECC_G_STR *p_ecc_para, SM2_CRYPT_CTX* sm2_context, SM3_CTX * sm3_context, UINT8 *plain, UINT32 byte_len, UINT8 pubkey[64], UINT8 C1[64], UINT8 *C2, UINT8 C3[32]); </pre>	
功能描述	SM2 加密运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		SM2_CRYPT_CTX* sm2_context: SM2_CRYPT_CTX 结构指针
		SM3_CTX * sm3_context: SM3_CTX 结构指针
		UINT8 *plain: 加密数据的起始地址
		UINT32 byte_len: 加密数据的字节长度
		UINT8 pubkey[64]: 公钥的起始地址
	输出	UINT8 C1[64]: C1 的起始地址
		UINT8 *C2: C2 的起始地址
		UINT8 C3[32]: C3 的起始地址
	返回值	0：运算成功；其他：运算失败

1.9.3.4 SM2 解密运算

函数形式	<pre> UINT8 sm2_decrypt(ECC_G_STR *p_ecc_para, SM2_CRYPT_CTX* sm2_context, SM3_CTX * sm3_context, UINT8 prikey[32], UINT8 C1[64], UINT8 *C2, UINT8 C3[32], UINT32 byte_len, UINT8 *plain, UINT32 mode); </pre>	
功能描述	SM2 解密运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		SM2_CRYPT_CTX* sm2_context: SM2_CRYPT_CTX 结构指针
		SM3_CTX * sm3_context: SM3_CTX 结构指针

		UINT8 prikey [64]: 私钥的起始地址
		UINT8 C1[64]: C1 的起始地址
		UINT8 *C2: C2 的起始地址
		UINT8 C3[32]: C3 的起始地址
		UINT32 byte_len: 密文数据的字节长度
		UINT8 mode: 模式功能选择, 0xF27E4B40: 普通模式 其他数据 : 安全模式
	输出	UINT8 *plain: 解密数据的起始地址
	返回值	0: 解密成功; 其他: 解密失败

1.9.3.5 SM2 密钥交换运算

函数形式	<pre>int sm2_Exchange_Key(ECC_G_STR *p_ecc_para, UINT8 role, UINT8 *Prikey, UINT8 *PubkeyB, UINT8 *Prikey_temp, UINT8 *Pubkey_temp, UINT8 *Pubkey_tempB, UINT8 *ZA, UINT8 *ZB, UINT32 klen, UINT8 *Ex_K, UINT8 *S1, UINT8 *SA);</pre>	
功能描述	SM2 密钥交换运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		UINT8 role: 密钥交换角色 1: 发起者 0: 接受者
		UINT8 *Prikey: 自身私钥的起始地址
		UINT8 *PubkeyB: 对方公钥的起始地址
		UINT8 *Prikey_temp: 自身临时私钥的起始地址
		UINT8 *Pubkey_temp: 自身临时公钥的起始地址
		UINT8 *Pubkey_tempB: 对方临时公钥的起始地址
		UINT8 *ZA: 自身的 Z 值
		UINT8 *ZB: 对方的 Z 值
		UINT32 klen: 交换密钥的位长度
	输出	UINT8 *Ex_K: 交换密钥的起始地址
		UINT8 *S1: S1 的起始地址

		UINT8 *SA: SA 的起始地址
	返回值	0: 交换成功; 其他: 交换失败

1.9.3.6 SM3 运算

函数形式	void sm3_hash(UINT8 *pDataIn,UINT32 DataLen,UINT8 *pDigest);	
功能描述	SM3 运算	
参数说明	输入	UINT8 *pDataIn: 待压缩数据的起始地址
		UINT32 DataLen: 待压缩数据的字节长度
	输出	UINT8 *pDigest: 摘要值的起始地址
	返回值	无

1.9.3.7 SM3 运算-初始化

函数形式	void SM3_initial (SM3_CTX *context);	
功能描述	SM3 运算-初始化	
参数说明	输入	无
	输出	SM3_CTX *context: SM3_CTX 结构指针
	返回值	无

1.9.3.8 SM3 运算-更新

函数形式	void SM3_update (SM3_CTX *context, UINT8 *input,UINT32 inputLen);	
功能描述	SM3 运算-更新	
参数说明	输入	SM3_CTX *context: SM3_CTX 结构指针 (运算前)
		UINT8 *input: 输入数据起始地址
		UINT32 inputLen: 输入数据字节长度
	输出	SM3_CTX *context: SM3_CTX 结构指针 (运算后)
	返回值	无

1.9.3.9 SM3 运算-结束

函数形式	void SM3_final (UINT8 *digest, SM3_CTX *context);	
功能描述	SM3 运算-结束	
参数说明	输入	SM3_CTX *context: SM3_CTX 结构指针 (运算前)
	输出	UINT8 *digest: 摘要值的起始地址
	返回值	无

1.9.4 注意事项及数据格式

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟, HASH 模块时钟使能;
- 2) 单独使用 SM3/SHA1/SHA256 模块则只需要确保模块时钟使能寄存器中的 UAC,HASH

模块时钟使能；

- 3) 通过时钟分频寄存器配置好系统时钟和算法时钟；
- 4) 在调用签名函数（SM2_sign）时，要保证输入参数 *hashdata* 的字长度为 CurveLength，输出参数 *Signature0*，*Signature1* 的字长度为 CurveLength，并初始化为 0；
- 5) 函数注释中的 big endian 表示高位字节数据存放在数组的低位，其中涉及到 SM3 运算的数据格式也是 big endian；

若：A = 0x11223344556677889900；

则：A[0] = 0x11；

A[1] = 0x22；

A[2] = 0x33；

A[3] = 0x44；

A[4] = 0x55；

A[5] = 0x66；

A[6] = 0x77；

A[7] = 0x88；

A[9] = 0x99；

A[10] = 0x00；

1.10 HASH

1.10.1 主要特性

- 支持 SHA1/SHA256/SHA384/SHA512 算法；

1.10.2 库需求说明

HASH 库需要的头文件如下：

sha1.lib	实现 sha1 运算
sha1.h	sha1.lib 对应的头文件
sha256.lib	实现 sha256 运算
sha256.h	sha256.lib 对应的头文件
sha384.lib	实现 sha384_512 运算
sha384.h	sha384.lib 对应的头文件

SHA1 库需要的资源如下：

库大小	88
全局变量	无
堆栈大小	384（字节）

SHA256 库需要的资源如下：

库大小	40
全局变量	无
堆栈大小	428（字节）

SHA384_512 库需要的资源如下：

库大小	1152
全局变量	无
堆栈大小	964(字节)

1.10.3 函数说明

1.10.3.1 SHA1 运算

函数形式	void SHA1_hash(UINT8 *pDataIn,UINT32 DataLen,UINT8 *pDigest);	
功能描述	SHA1 运算	
参数说明	输入	UINT8 *pDataIn: 待压缩数据的起始地址
		UINT32 DataLen: 待压缩数据的字节长度
	输出	UINT8 *pDigest: 摘要值的起始地址
	返回值	无

1.10.3.2 SHA1 运算-初始化

函数形式	void SHA1_init (SHA1_CTX *context);	
功能描述	SHA1 运算-初始化	
参数说明	输入	无
	输出	SHA1_CTX *context: SHA1_CTX 结构指针
	返回值	无

1.10.3.3 SHA1 运算-更新

函数形式	void SHA1_update (SHA1_CTX *context,UINT8 *input,UINT32 inputLen);	
功能描述	SHA1 运算-更新	
参数说明	输入	SHA1_CTX *context: SHA1_CTX 结构指针（运算前）
		UINT8 *input: 输入数据起始地址
		UINT32 inputLen: 输入数据字节长度
	输出	SHA1_CTX *context: SHA1_CTX 结构指针（运算后）

	返回值	无
--	-----	---

1.10.3.4 SHA1 运算-结束

函数形式	void SHA1_final (UINT8 *digest, SHA1_CTX *context);	
功能描述	SHA1 运算-结束	
参数说明	输入	SHA1_CTX *context: SHA1_CTX 结构指针 (运算前)
	输出	UINT8 *digest: 摘要值的起始地址
	返回值	无

1.10.3.5 SHA256 运算

函数形式	void SHA256_hash(UINT8 *pDataIn, UINT32 DataLen, UINT8 *pDigest);	
功能描述	SHA256 运算	
参数说明	输入	UINT8 *pDataIn: 待压缩数据的起始地址
		UINT32 DataLen: 待压缩数据的字节长度
	输出	UINT8 *pDigest: 摘要值的起始地址
	返回值	无

1.10.3.6 SHA256 运算-初始化

函数形式	void SHA256_init (SHA256_CTX *context);	
功能描述	SHA256 运算-初始化	
参数说明	输入	无
	输出	SHA256_CTX *context: SHA256_CTX 结构指针
	返回值	无

1.10.3.7 SHA256 运算-更新

函数形式	void SHA256_update (SHA256_CTX *context, UINT8 *input, UINT32 inputLen);	
功能描述	SHA256 运算-更新	
参数说明	输入	SHA256_CTX *context: SHA256_CTX 结构指针 (运算前)
		UINT8 *input: 输入数据起始地址
		UINT32 inputLen: 输入数据字节长度
	输出	SHA256_CTX *context: SHA256_CTX 结构指针 (运算后)
	返回值	无

1.10.3.8 SHA256 运算-结束

函数形式	void SHA256_final (UINT8 *digest, SHA256_CTX *context);	
功能描述	SHA256 运算-结束	
参数说明	输入	SHA256_CTX *context: SHA256_CTX 结构指针 (运算前)
	输出	UINT8 *digest: 摘要值的起始地址
	返回值	无

1.10.3.9 SHA384 运算

函数形式	void SHA384_hash(UINT8 *pDataIn, UINT32 DataLen, UINT8 *pDigest);	
功能描述	SHA384 运算	
参数说明	输入	UINT8 *pDataIn: 待压缩数据的起始地址
		UINT32 DataLen: 待压缩数据的字节长度
	输出	UINT8 *pDigest: 摘要值的起始地址
	返回值	无

1.10.3.10 SHA384 运算-初始化

函数形式	void SHA384_init (SHA384_CTX *context);	
功能描述	SHA384 运算-初始化	
参数说明	输入	无
	输出	SHA384_CTX *context: SHA384_CTX 结构指针
	返回值	无

1.10.3.11 SHA384 运算-更新

函数形式	void SHA384_update (SHA384_CTX *context, UINT8 *input, UINT32 inputLen);	
功能描述	SHA384 运算-更新	
参数说明	输入	SHA384_CTX *context: SHA384_CTX 结构指针 (运算前)
		UINT8 *input: 输入数据起始地址
		UINT32 inputLen: 输入数据字节长度
	输出	SHA384_CTX *context: SHA384_CTX 结构指针 (运算后)
	返回值	无

1.10.3.12 SHA384 运算-结束

函数形式	void SHA384_final (UINT8 *digest, SHA384_CTX *context);	
功能描述	SHA384 运算-结束	
参数说明	输入	SHA384_CTX *context: SHA384_CTX 结构指针 (运算前)
	输出	UINT8 *digest: 摘要值的起始地址
	返回值	无

1.10.3.13 SHA512 运算

函数形式	void SHA512_hash(UINT8 *pDataIn, UINT32 DataLen, UINT8 *pDigest);	
功能描述	SHA512 运算	
参数说明	输入	UINT8 *pDataIn: 待压缩数据的起始地址
		UINT32 DataLen: 待压缩数据的字节长度
	输出	UINT8 *pDigest: 摘要值的起始地址
	返回值	无

1.10.3.14 SHA512 运算-初始化

函数形式	void SHA512_init (SHA384_CTX *context);	
功能描述	SHA512 运算-初始化	
参数说明	输入	无
	输出	SHA384_CTX *context: SHA384_CTX 结构指针
	返回值	无

1.10.3.15 SHA512 运算-更新

函数形式	void SHA384_update (SHA384_CTX *context,UINT8 *input,UINT32 inputLen);	
功能描述	SHA512 运算-更新	
参数说明	输入	SHA384_CTX *context: SHA384_CTX 结构指针 (运算前)
		UINT8 *input: 输入数据起始地址
		UINT32 inputLen: 输入数据字节长度
	输出	SHA384_CTX *context: SHA384_CTX 结构指针 (运算后)
	返回值	无

1.10.3.16 SHA512 运算-结束

函数形式	void SHA512_final (UINT8 *digest, SHA384_CTX *context);	
功能描述	SHA512 运算-结束	
参数说明	输入	SHA384_CTX *context: SHA384_CTX 结构指针 (运算前)
	输出	UINT8 *digest: 摘要值的起始地址
	返回值	无

1.10.4 注意事项及数据格式

- 1) 需要确保模块时钟使能寄存器中的 UAC，HASH 模块时钟使能;
- 2) 通过时钟分频寄存器配置好系统时钟和算法时钟;
- 3) 涉及到 HASH 运算的数据格式都是 big endian;

若: A = 0x11223344556677889900;

则: A[0] = 0x11;

A[1] = 0x22;

A[2] = 0x33;

A[3] = 0x44;

A[4] = 0x55;

A[5] = 0x66;

A[6] = 0x77;
A[7] = 0x88;
A[9] = 0x99;
A[10] = 0x00;

1.11 ECDSA

1.11.1 主要特性

- 支持 160~521 比特的任意素域上的 ECDSA 密钥生成，签名验证算法;

1.11.2 库需求说明

ECDSA 库需要的头文件如下：

ecdsa.lib	实现 ECDSA 相关运算
ecdsa.h	ecdsa.lib 对应的头文件

ECDSA 库需要的资源如下：

库大小	960（字节）
全局变量	无
堆栈大小	948（字节）

1.11.3 函数说明

1.11.3.1 ECDSA 签名运算

函数形式	UINT8 ECDSA_sign(ECC_G_STR *p_ecc_para, MATH_G_STR *p_math_str, UINT32 *hashdata, UINT32 *PrivateKey, UINT32 *Signature0, UINT32 *Signature1);	
功能描述	ECDSA 签名运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针

		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 *hashdata: 摘要的起始地址
		UINT8*PrivateKey: 私钥的起始地址
	输出	UINT8 *Signature0: 签名结果 r 的起始地址
		UINT8 *Signature1: 签名结果 s 的起始地址
	返回值	0: 运算成功; 1: 运算失败

1.11.3.2 ECDSA 验签运算

函数形式	<pre>ECDSA_verify(ECC_G_STR *p_ecc_para, MATH_G_STR *p_math_str, UINT32 *hashdata, UINT32 *PublicKeyX, UINT32 *PublicKeyY, UINT32 *Signature0, UINT32 *Signature1);</pre>	
功能描述	ECDSA 验签运算	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
		MATH_G_STR *p_math_st: MATH_G_STR 结构指针
		UINT8 *hashdata: 摘要的起始地址
		UINT8* PublicKey: 公钥的起始地址
		UINT8 *Signature0: 签名 s 的起始地址
		UINT8 *Signature1: 签名 r 的起始地址
	输出	无
	返回值	0: 验签成功; 1: 验签失败

1.11.3.3 ECDSA 密钥生成

函数形式	<pre>void ECDSA_keypair(ECC_G_STR *p_ecc_para, UINT32 PrivateKey[], UINT32 PublicKeyX[], UINT32 PublicKeyY[]);</pre>	
功能描述	生成 ECDSA 密钥	
参数说明	输入	ECC_G_STR *p_ecc_para: ECC_G_STR 结构指针
	输出	UINT32 PrivateKey[]: 私钥的起始地址
		UINT32 PublicKeyX[]: 公钥 X 的起始地址
		UINT32 PublicKeyY[]: 公钥 Y 的起始地址
	返回值	无

1.11.4 注意事项及数据格式

- 1) 需要确保模块时钟使能寄存器中的 UAC, PKI, HRNG, HRNG 慢时钟模块时钟使能;
- 2) 通过时钟分频寄存器配置好系统时钟和算法时钟;
- 3) 在调用签名函数(**ECDSA_sign**)时,要保证输入参数 *hashdata* 的字长度为 CurveLength, 输出参数 *Signature0*, *Signature1* 的字长度为 CurveLength, 并初始化为 0;
- 4) ECDSA 的所有数据存放方式如下:

低 32 位字数据在数组的低位, 每个字以大端方式存放。

若: $A = 0x11223344556677889900$;

则: $A[0] = 0x77889900$;

$A[1] = 0x33445566$;

$A[2] = 0x00001122$;

1.12 算法库使用注意事项

1、如果需要将算法 sram 作为普通 sram 使用, 请先调用 `alg2nor_sram()` 函数, 具体见 `release/pki/demo` 工程。

2、`release` 中提供的算法库是通用库, 如果需要去检测机构进行测试, 则需要与芯片厂家联系获取在该检测机构备案的算法库。